Marx Generator: Technical Implementation Specification

Version 1.3 · 10 October 2026 · Status: implementation contract; hardware and measured-data verification pending.

This document specifies a transparent backend for recommending the three best distinct evaluated Marx-generator setups. It combines the project's latest architecture decisions with the supplied physics formula book. The frontend will be designed later. Every implementation module below has explicit inputs, outputs, constraints, standalone fixtures and acceptance criteria, so an AI agent can implement it without waiting for adjacent modules.

1. Decisions and authority

1.1 Committed architecture

Component	Responsibility	Selected method
Physics	Initial inverse estimates, forward waveform baseline, electrical stress calculations	Formula-book calculator mode for regression; topology-specific analytic/ODE mode for circuit predictions
Assembly generation	Enumerate buildable front and tail resistor networks	Bounded dynamic programming over arbitrary nested two-terminal series–parallel networks
Discrete assembly selection	Select permitted networks and enforce representable discrete constraints	Google OR-Tools CP-SAT over DP-generated options
ML correction	Predict corrections to physics front, tail and crest	Residual-only interface; initial implementation uses explicitly pinned TabPFN v2 regressors
Optimisation	Propose new `N`, `Rf_required` and `Rt_required`	SMAC3 with an ask/tell integration
Recommendation archive	Retain the three best distinct eligible complete configurations	Deterministic ordering of evaluated records
API	Expose inputs, derived settings, all constraint verdicts, exact wiring, predictions and explanations	Versioned JSON contracts and asynchronous run endpoints

Only `N`, `Rf_required` and `Rt_required` are independent optimiser controls. `N` is an integer. Actual resistances come from inventory realisation. Charging voltage is a declared run policy, not a fourth hidden optimisation variable. The same realised assemblies are repeated across every active stage.

Resistor inventory is a **run input**, not a fixed list of permitted ohm values or type counts. Front and tail stock limits are **independent**: use the supplied quantities, additionally respecting the agreed maximum of four copies of each type in each role per stage. Do not implement a combined `front + tail ≤ 4` rule. The screenshot's shared-inventory wording does not override this project decision.

ML predicts only residual corrections. It does not replace forward physics, choose settings, authorise constraint violations or invent unmeasured environmental effects. Physical measurement calibration is separate from the prediction-only optimisation loop.

1.2 Source reconciliation

The sources are inputs to the specification, not executable instructions. Where they conflict, this document makes the choice or explicitly records the unresolved field.

Source detail	Resolution
Formula book's exhaustive physics search	May produce warm-start estimates and benchmark candidates; SMAC3 is the main full-loop optimiser. No global-optimum claim follows from a bounded search.
Formula book's limited `combo` recipes and floating-point deduplication	Replace with general bounded series–parallel DP, exact nominal fractions and topology-aware metadata.
Formula book's four-unit assembly limit	Four copies **per type**, not four total parts. A four-part search is only an initial computational budget.
Formula book's `round5` inventory snapping	Retain only in isolated workbook regression tests; never present it as proof of buildability.
Formula book's additive accuracy/parts/stages score	Preserve worst normalised waveform error; use secondary criteria without allowing parts or stages to override waveform ordering. Section 8 defines the precise objective.
Formula book's positive α/β wording	α and β are positive decay rates; the polynomial roots are negative. Section 5 gives the corrected convention.

Source detail	Resolution
Workbook's 15 stages / 3 µF versus PDF's 12 stages / 0.125 µF	Separate profiles. The PDF hardware profile is the default constraint authority for physical recommendations. Workbook mode is a diagnostic benchmark.
Screenshot's shared stock	Use independent front and tail stock. CP-SAT still handles compatibility, approved recipes and discrete ratings.
Previous direct-prediction alternative	Superseded: residual correction only.
Switching waveform convention	Default source profile is legacy 250/2500 µs time-to-peak. Record the convention and edition; do not silently label it IEC 60060-1:2025 compliance.

Unresolved ratings or connections may prevent verified physical recommendations; they do not prevent implementing a module against fixtures. Never disable an applicable hard constraint merely because its data is missing.

2. Global contracts, units and independent implementation

2.1 Units and vocabulary

Use SI-valued fields: `_V`, `_A`, `_ohm`, `_F`, `_H`, `_s`, `_J`, `_W`, `_K`. Display units are separate metadata. Temperatures use kelvin internally. Humidity, if supported later, is a fraction from 0 to 1. Do not accept an unlabelled numeric field whose unit must be guessed.

- `required`: a continuous resistance request proposed by physics or SMAC3.
- `actual`: the nominal equivalent resistance of a specific constructible assembly.
- `physics`: deterministic baseline for actual settings, including its fidelity and topology.
- `residual`: ML-estimated observed-minus-physics difference, in each output's SI unit.
- `corrected`: physics plus residual.
- `measured`: laboratory evidence, never a synonym for a corrected prediction.
- `supported`: operating point falls within a declared validated model domain.
- `hardware_feasible`: every required hardware verdict is `pass`.
- `waveform_within_tolerance`: all three corrected waveform errors meet their bounds.

Every serialisable record includes `schema_version`, `record_id`, creation time, versions/hashes of relevant sources and status. Reject NaN and infinity. Use `null` plus an explanatory status for unavailable outputs.

2.2 Core record types

Record	Required fields
`RunRequest`	Equipment/test-profile IDs; inline inventory or registered inventory reference; system voltage; required crest; load/setup; environment with availability; expected efficiency; charging policy; search budgets; DP resistance tolerances; objective configuration; model version; seed
`ResolvedRunContext`	Full immutable snapshot of request, defaults, hardcoded hardware values, source provenance, topology, waveform definitions and model feature manifest
`Proposal`	`N`, `Rf_required_ohm`, `Rt_required_ohm`, proposal source, proposal index, SMAC trial metadata
`Assembly`	Role, exact nominal resistance, topology tree, explicit netlist, leaf IDs, per-type quantities, assembly counts, approvals, ratings metadata and search guarantee
`RealisedCandidate`	Proposal, actual front/tail assemblies, `U_charge_V`, `N`, stage topology, per-stage replication manifest, canonical identity
`PhysicsEvaluation`	Effective circuit parameters; waveform metrics/trace; efficiency; stage energies; component stresses; assumptions; numerical diagnostics; constraint report
`CorrectionEvaluation`	Baseline ID/hash; feature values/provenance; three residuals; corrected outputs; model support; validated uncertainty if available; explanation records
`ObjectiveEvaluation`	Signed/absolute/percentage/normalised errors; worst error; weighted mean; optimiser cost; archive key; tolerance flags
`CandidateEvaluation`	Full nested realised/physics/correction/objective records; eligibility; all constraints; durations; rejection reasons
`WaveformPlotBundle`	Target reference samples, one ML-parameter reconstruction per returned candidate, common SI time grid, curve provenance, metrics, extraction checks and plot metadata
`RunResult`	Ordered top-three records; waveform plot bundle/reference; history/query references; counters; stop reason; shortfall reason; guarantee scope; model/data/hardware versions

2.3 Universal constraint report

Use the same structure at every module boundary:

```
{
  "constraint_id": "HW_STAGE_ENERGY",
  "origin": "problem_statement_page_4",
  "source_value": 2500,
  "unit": "J",
  "kind": "hard",
  "enforced_by": ["physics", "final_eligibility"],
  "rule": "stage_energy_J <= 2500",
  "observed_value": 625,
  "limit_value": 2500,
  "margin": 1875,
  "status": "pass",
  "reason_code": null,
  "evidence_record_ids": ["fixture-stage-energy-1"]
}
```

Statuses are `pass`, `fail`, `unknown`, `not_applicable`. Include a reason for `unknown` and `not_applicable`. Hard `unknown` is not a pass; aggregate hardware status is unresolved. Expose checks skipped because an earlier module failed. Metadata such as height is retained but not converted into a fabricated pass/fail electrical bound.

2.4 Contract-first development rule

Each module must accept explicit records and return explicit records; never read an implicit global “previous stage”. Commit schemas, fixtures and interface protocols before implementing integrations. Adjacent modules can be replaced with deterministic test doubles. A fixture must be labelled `mock` or `diagnostic`, and its results must never appear as measured or hardware-verified evidence.

Suggested ownership:

```
contracts/       Typed records, JSON schemas, constraint IDs, fixtures
profiles/        Hardware, inventory and waveform profiles with provenance
physics/         inverse, forward, waveform, stress, plot_reconstruction
assembly/        DP, canonicalisation, CP-SAT selection
correction/      data, features, residual model, support, explanations
optimisation/    objective, SMAC adapter, budgets, top-three archive
api/             validation, routes, jobs, events, serialisation
validation/      module tests, contract tests, integration fixtures
```

Module work packages in Sections 4–10 each specify local acceptance tests. Section 11 defines the public API. No component may import an implementation from another component merely to obtain a test fixture.

3. Inputs and hardcoded constraints

3.1 User/scenario inputs

All original calculator inputs remain considered even though only three controls are optimised.

API field	Type / unit	Role
`system_max_voltage_V`	Positive number, V	Equipment/system voltage classification; not crest or a charge ceiling
`impulse_profile_id`	Versioned ID	Targets, extraction convention and tolerances
`target.crest_V`	Positive number, V	Required test crest
`load.model_id`	ID	Capacitive, RC, RLC or registered network model
`load.C_F`, `load.R_ohm`, `load.L_H`	Model-dependent numbers	Actual load parameters; optional only when the selected model does not require them
`load.topology`, `load.terminal_connections`	Structured network / mapping	Distinguishes series/parallel RLC and transformer tested/other terminals
`setup.divider_C_F`	Nonnegative number	Divider loading
`setup.stray_C_F`	Nonnegative number	Stray loading, with assigned circuit location

API field	Type / unit	Role
`setup.stray_L_H`	Nonnegative number	Loop/lead parasitic inductance, separate from load inductance
`setup.leads`, `setup.return_path`, `setup.geometry`	Structured metadata	Declared physical configuration; model uses it only through supported parameters
`setup.basic_capacitance_inclusion`	Allocation manifest	Declares whether the 545 pF basic capacitance is separate or already included
`expected_voltage_efficiency`	Fraction `(0,1]`	Initial sizing estimate; never silently treated as measured per-stage efficiency
`minimum_voltage_efficiency`	Optional fraction	Explicit additional acceptance requirement if requested; no default target invented
`environment`	Values, units, sensor/time/source and missing flags	Feature set to be finalised with measured data
`charging_policy`	Discriminated object	Fixed charge default; optional explicitly implemented deterministic derived-charge policy
`equipment_profile_id`	ID	Resolve protected fixed equipment ratings; does not fix the resistor catalogue
`inventory`	Inline front/tail catalogues or a registered catalogue reference	Required input defining actual available values, quantities, scope, ratings and approvals for this run
`dp_tolerances`, `search`, `objective`, `model_version`	Structured values	Declared run policy, not hidden algorithm choices

The workbook's example values are 400 kV system voltage; lightning; 1,425 kV crest; 850/500/150 pF object/divider/stray capacitance; 18.5 µH stray inductance; expected efficiency 0.82; maximum 15 stages; maximum 200 kV/stage; 3 µF/stage. The last three are equipment-profile settings, not mutable SMAC variables. Return their resolved values even when the client supplied only a profile ID.

Expected generator voltage-transfer efficiency `V_crest/(N U_charge)` is different from voltage utilisation `U_charge/U_stage,max`. An energy efficiency is another quantity. If per-stage efficiency is later required, define and record its measurement/model separately; do not derive it from the single workbook efficiency field.

3.2 Mandatory hardcoded source parameters

Implementation requirement: use all fixed generator constraints and parameters listed on the final two pages of the problem statement, including those absent from the request input fields. Store them in a protected versioned equipment profile, inject them into every resolved run context, and expose their values and per-stage enforcement outcomes through the API. Relevant fixed electrical values are hard constraints or model constants; installation metadata and unspecified entries remain explicitly represented. They are not silently ignored because the client did not send them.

Default physical equipment profile: `haefely_pdf_v1`. Immutable hardware values are:

ID / parameter	Value	Role / enforcement
`HW_STACK_CHARGE`	2,400,000 V	Sum of charged stage voltages / ideal erected voltage ceiling for approved series arrangement
`HW_TOTAL_ENERGY`	30,000 J	Total stored-energy ceiling
`HW_STAGE_COUNT`	Minimum 2, maximum 12	Integer active-stage bound
`HW_STAGE_CHARGE`	200,000 V/stage	Initial capacitor/stage charge ceiling
`HW_STAGE_ENERGY`	2,500 J/stage	Stored-energy ceiling for each stage
`C_stage_F`	`1.25e-7` F (0.125 µF)	Fixed stage capacitance
`REQ_CREST_RANGE`	50,000–2,400,000 V	Permitted request range; not proof of achievable crest
`standard_waveforms`	Lightning 1.2/50 µs; switching 250/2500 µs	Source timing profiles; preserve definition
`pdf_front_resistors`	30 Ω, 465 Ω, 3,700 Ω	Source equipment resistor values; keep separate from revised project inventory
`pdf_tail_lightning`	520 Ω	Source lightning tail resistor
`pdf_tail_switching`	22,000 Ω	Source switching tail resistor

ID / parameter	Value	Role / enforcement
`charging_resistor_RL`	Table: 135,000 Ω; embedded nameplate: 435,000 Ω	Conflict unresolved; store both readings, resolved value `null` pending verification
`external_front_Rse1_Rse2_Rse3`	Unspecified, shown as dash	Each field `null`, with fitted/state/location verification required
`external_tail_Rpe`	Unspecified, shown as dash	`null`, not zero
`potential_Rpot`	2,000,000 Ω	Include loading in the applicable active circuit branch
`discharge_Rerd1`	5,450 Ω	Discharge/interlock branch parameter, with operating-state declaration
`discharge_Rerd2`	13,000 Ω	Discharge/interlock branch parameter, with operating-state declaration
`HW_SHOT_RATE`	2 physical impulses/minute	Laboratory acquisition limit; separate from software iterations
`C_basic_F`	`5.45e-10` F (545 pF)	Basic circuit loading; include once at verified location
`sphere_diameter_m`	0.25 m	Diameter metadata; not gap spacing or a breakdown-voltage setting
`stage_connections`	Source example `1p12s`; other fields incomplete	Approved arrangements required; no arbitrary stage-topology assumption
`height_m`	Approximately 8.7 m	Fixed installation metadata relevant to setup verification
`base_area`, `total_weight`, `heaviest_part_weight`	Unspecified, shown as dash	Retain `null` and source status; do not invent numerical constraints

Page 3 additionally requires considering variable load/divider/stray capacitances, stray inductance, leads, geometry and grounding/return-path configuration. Page 4 requests measured waveform data for ML calibration and validation. These are recorded scenario/data requirements, not replaceable by a fictitious fixed environment.

This profile does not silently claim unresolved charging/external branches have been verified. A verified circuit phase may mark a branch `not_applicable` with equipment evidence; otherwise a recommendation depending on it remains unresolved. Missing component pulse ratings and transient/reverse capacitor ratings are also unresolved hard checks when applicable.

3.3 Inventory input contract and example catalogues

Accept any positive finite nominal resistor values, any number of front/tail types, and explicitly supplied nonnegative integer quantities. There is no requirement to contain 10 front or 12 tail types, or to match the workbook's values. The two catalogues below are **example input profiles only**. PDF resistor values are recorded source equipment data, not a universal allowlist for incoming inventory.

The request supplies exactly one of:

```
{
  "inventory": {
    "source": "inline",
    "front": [
      {"type_id": "front_37p5", "nominal_ohm": 37.5, "available_quantity": 3, "quantity_scope":
"per_stage", "tolerance_fraction": 0.01},
      {"type_id": "front_100", "nominal_ohm": 100, "available_quantity": 4, "quantity_scope": "per_stage",
"tolerance_fraction": 0.01}
    ],
    "tail": [
      {"type_id": "tail_7p5", "nominal_ohm": 7.5, "available_quantity": 4, "quantity_scope": "per_stage",
"tolerance_fraction": 0.01},
      {"type_id": "tail_60", "nominal_ohm": 60, "available_quantity": 2, "quantity_scope": "per_stage",
"tolerance_fraction": 0.01}
    ]
  }
}
```

or `inventory={"source":"registered","profile_id":"project_front10_tail12_v1","version":"1"}`. Inline values are snapshotted and hashed exactly like registered profiles. Reject both sources being supplied simultaneously, malformed values and duplicate role-specific type IDs. Types with zero quantity remain visible but are excluded from the search. An empty usable catalogue in either required role is a

domain failure. Equal nominal values with different ratings or type identities must not be merged unless their interchangeability is explicitly established.

Each type exposes `type_id`, `nominal_ohm`, `available_quantity`, `quantity_scope` (`per_stage` or `global`), optional manufacturing tolerance, ratings/thermal fields, component identifiers where available and approval evidence. Missing ratings are unknown, not implicit unlimited ratings. Nominal values, number of types and available quantities are client-supplied inputs; equipment limits and approved wiring remain enforced separately.

The previously agreed four-per-type-per-role stage ceiling remains a hardware/project policy. Supplied stock can contain more than four units, but the effective per-stage quantity for proposal `N` is:

```
q_effective[type, N] = min(4, available_quantity)               if per_stage
q_effective[type, N] = min(4, floor(available_quantity / N))    if global
```

Thus two supplied units allow at most two, and ten supplied units do not authorise more than four of that type in a stage. Front and tail limits are calculated independently. A future equipment profile changing this ceiling must explicitly version that hardware policy rather than infer it from stock abundance. The inventory is fixed within a run; SMAC optimises settings, not stock. Changing the catalogue starts a new context/run or requires explicit reevaluation with a new inventory hash.

Workbook example input profiles:

Role	Ω values	Quantity per type per stage	Total listed units
Front	10, 20, 30, 50, 75, 100, 150, 200, 300, 500	4 independently	40
Tail	5, 10, 15, 20, 25, 30, 40, 50, 75, 100, 150, 200	4 independently	48

Each type has a role-specific `type_id`, nominal value, tolerance, quantity scope, pulse-voltage/current/energy ratings, thermal parameters if known, approved usage and evidence. A front 30 Ω type and a tail 30 Ω type are separate stock identities. Every stage repeats the same selected pair; global warehouse stock, if recorded, is an additional `N × per_stage_quantity` check within each role.

Maintain PDF resistor values as source provenance while resolving the active catalogue entirely from the inventory input. Record its source, hash and verification status; do not erase the PDF values or silently substitute them for the supplied stock. Hardware approval depends on actual unit ratings and permitted connections, not on membership in the workbook list.

For the workbook example only, the maximum series equivalent from all front parts is `4 × 1,435 = 5,740 Ω`; for tail it is `4 × 720 = 2,880 Ω`. For positive resistor-only series–parallel networks no equivalent exceeds the all-series sum. Therefore the workbook switching example's approximately 14,257 Ω front requirement is outside that example catalogue's complete upper bound. This conclusion does not apply to another supplied inventory. Report `inventory_range_infeasible` when a required interval lies beyond proven bounds. Do not fabricate a resistor or silently retain `round5` as an assembly.

For each supplied role and proposed stage count, derive `R_max=sum(q_effective_i*R_i)` and `R_min=1/sum(q_effective_i/R_i)` over usable types. These are full-stock all-series/all-parallel bounds for unrestricted positive resistor-only series–parallel wiring; approved recipe restrictions or a smaller part budget may tighten them. An interval overlapping these bounds still need not contain an achievable network. Recompute bounds, DP dimensions, quantity limits and any SMAC search-space settings from the actual input. With global stock the bounds depend on `N`; expose this dependency and never reuse the workbook's 5,740/2,880 Ω constants as universal bounds.

3.4 Hardware checks for every realised candidate

```
N is integer and 2 <= N <= 12                          [PDF profile]
0 < U_charge_V <= 200000
sum(stage_charge_V) <= 2400000
E_stage[i] = 0.5 * C_stage[i] * U_charge[i]^2 <= 2500
sum(E_stage[i]) <= 30000
front_count[type, stage] <= q_effective_front[type,N] [supplied front stock, cap 4]
tail_count[type, stage] <= q_effective_tail[type,N]   [supplied tail stock, cap 4]
actual wiring is permitted; all used component ratings pass
active topology and all required branch values are resolved
candidate is within declared ML support before ML recommendation
```

Internal transient/reversal voltages, resistor pulse stress, insulation and clearances require their own verified bounds. Three predicted waveform parameters or a lumped single-capacitor model cannot certify every internal stress. Report `unknown` when the model or ratings are insufficient.

`workbook_benchmark_v1` uses 15 stages and 3 µF solely for calculator regression. Its missing energy ratings are unknown, not waived for physical use. At 200 kV, one 3 µF stage stores 60 kJ, incompatible with the PDF's 2.5 kJ stage limit. API benchmark results have `physical_recommendation_eligible=false`.

4. Work package A: Profiles, validation and immutable run context

Standalone inputs: `RunRequest`, local profile registry and schemas. **Outputs:** `ResolvedRunContext`, normalised SI fields, validation errors and full source/constraint manifest. **Interface:** `resolve_request(request, registry) -> ResolutionResult`.

Implement strict discriminated schemas. Validate electrical inputs against the chosen load model rather than requiring every R/L/C for a pure capacitor. Require positive effective generator/load capacitances for the base equations; allow zero stray inductance. Preserve auxiliary branch states, source conflicts and basic-capacitance allocation. Nonuniform stages are not supported by this first uniform-stage implementation.

Freeze a deep immutable context and hash it. No component can mutate hardcoded limits, environment, selected waveform definition or ML version during a run. New measured conditions start a new run. Permit a stricter client limit, but reject a client request to raise the profile's hardware limit. Return both the request value and effective bound.

Charging policy:

1. `fixed`: use a supplied validated `U_charge_V` throughout optimisation. This is the default integration policy.
2. `seed_then_fixed`: derive initial charge from the initial physics stage estimate and expected efficiency, validate it and freeze it before SMAC starts. Persist the derivation and frozen value.
3. Other deterministic recomputation policies require a registered implementation and version. They still do not add a free SMAC control. Expose the resulting charge for every proposal.

Avoid recomputing exact-model charge simply to force physics crest to the target on every candidate unless that is explicitly the desired operating policy: it changes the optimisation problem and can make the baseline crest objective constant by construction.

Independent fixtures: valid PDF-profile request with `N=9`, fixed charge 100,000 V, basic capacitance explicitly added; invalid requested `N=13`; invalid 200,001 V charge; benchmark profile; unresolved branch conflict. Fixtures can use declared mock topology approvals.

Acceptance: exact SI conversion; all profile constants present in response; immutable context hash stable; no 15-stage workbook/PDF mixture; conflicting/missing values stay visible; four front plus four tail units of one nominal value accepted; client cannot bypass a hard bound; arbitrary inline inventory and registered inventory resolve identically; stock scopes and derived bounds follow the supplied data.

5. Work package B: Physics, inverse estimates and waveform extraction

Standalone inputs: context plus either a stage count for inverse estimates or a complete realised candidate for forward evaluation. **Outputs:** `IdealProposal` or `PhysicsEvaluation`. **Interfaces:** `initial_proposals(context)`, `inverse_for_N(context, N)`, `evaluate_actual(context, candidate)`.

5.1 Base topology and formula contract

The supplied book assumes tail resistance across the generator-side capacitance. Register this as `generator_side_tail_v1`. It must be verified against actual wiring; a load-side tail requires different equations.

For identical erected series stages:

```
C1 = C_stage / N
V0 = N * U_charge
R1 = N * Rf_actual + R_front_external_series
R2 = reduce_verified_tail_network(candidate, registered_topology)
C2 = C_load + C_divider + C_stray + C_basic_if_not_already_included
```

Summing capacitances is valid only for capacitances mapped to the same effective terminals. Arbitrary transformer/RLC networks must use a registered network solver and terminal contract. Do not substitute `L_load` for `L_stray` or add all inductances without checking topology.

Constants from the formula book for calculator regression are `k_front=1.67`, `k_LC=2.5`, `k_tail=0.693`. Record constants and fidelity in every evaluation.

5.1.1 Screenshot-aligned physics input and derived-feature contract

The supplied feature screenshots are adopted as an explicit interface, with two corrections: active stages satisfy **2 ≤ N ≤ 12** for the PDF equipment, and charging voltage follows the frozen declared policy rather than adding a fourth independent SMAC variable. The object-capacitance field generalises to all supported loads, not only insulators.

Physics field	Kind	Source / extraction rule
`generator_topology_id` and connection manifest	Nominal configuration	Verified registered generator arrangement
`C_stage_F`	Nominal equipment constant	PDF profile `1.25e-7 F`; benchmark profile separately uses `3e-6 F`

Physics field	Kind	Source / extraction rule
`N`	Realised candidate	Integer optimiser proposal, within profile bounds
`U_charge_V`	Candidate operating value	Supplied or derived by the declared charging policy; not an additional independent SMAC control
`C_object_F`	Nominal scenario variable	Measured/supplied `load.C_F` for a supported capacitive equivalent
`C_divider_F`	Nominal setup value	Supplied divider capacitance
`C_basic_F` and allocation manifest	Nominal setup constant	545 pF for the PDF profile, included exactly once at verified terminals
`C_stray_F`	Measured or estimated setup value	Supplied estimate, geometry model or prior calibration, with uncertainty/provenance
`L_effective_H`	Derived loop parameter	Verified loop reduction; distinguish generator, leads, return and component contributions
`C_generator_F`	Derived	`C_stage_F/N` for identical series-erected stages
`V_initial_erected_V`	Derived	`N*U_charge_V` for the approved series arrangement
`C_load_effective_F`	Derived	Sum of object/divider/basic/stray capacitances only where they share the effective terminals
`Rf_stage_actual_ohm`, `Rt_stage_actual_ohm`	Network-derived	Recursively reduce the exact selected per-stage assemblies
`R_front_effective_ohm`	Network-derived	`N*Rf_stage_actual + R_front_external_series` only for that registered series path
`R_tail_effective_ohm` or explicit tail network	Network-derived	Reduce the actual erected tail topology; do not universally assume `N*Rt_stage_actual`
Load R/L/network and terminal states	Model-dependent	Required for the selected general-load solver; not inferred from insulator capacitance
Active auxiliary branches	Circuit configuration	Include confirmed charging/potential/discharge/external branches in the relevant operating phase

The book's base topology has no external front resistor and assumes an N-series-equivalent tail resistance. For that named benchmark, `R_front_external_series=0` and `R2=N*Rt_actual`; preserve these regression equations. Other topologies must use their own reduction/forward solver. An external resistor with unknown presence/value is not automatically zero. If the inverse formula produces a nonpositive stage resistance after subtracting an external series resistor, reject that inverse proposal.

Screenshot-derived estimation formulas:

```
R_series = sum(R_i)
R_parallel = 1 / sum(1/R_i)
L_effective = sum(L_i) + 2*sum_over_i_less_j(s_i*s_j*M_ij)
C_stray = C_measured_total − C_known_distinct
```

The inductance expression applies to a consistently oriented common-current lumped loop; `s_i` specifies the current orientation and mutual terms are included where significant. General coupled networks require circuit reduction, not a scalar sum. For the workbook fixture, its `L_uH` field is interpreted as the supplied total loop estimate, so `L_effective=18.5e−6 H`; do not add the same lead inductance again.

The stray-capacitance subtraction must use the same measurement arrangement and non-overlapping known capacitances. Preserve measurement uncertainty. A negative estimate indicates inconsistent inputs or unresolved uncertainty; do not silently clamp it to zero. In the workbook regression profile, basic capacitance is intentionally omitted; do not mix its effective load with the PDF profile's basic-capacitance addition.

Each derived feature has value, unit, formula/version, source-field IDs, uncertainty/status and topology applicability. Expose the physics input manifest before forward evaluation, so an agent can verify extraction independently of the solver.

5.2 Calculator inverse and forward formulas

```
N_seed = max(N_min, ceil(V_target / (eta_expected * U_stage_max)))
U_seed = V_target / (N_seed * eta_expected)
D = (t_front_target / k_front)^2 − k_LC * L_effective * C2
R1_required = sqrt(D) / C2
```

```
R2_required = t_tail_target / (k_tail * (C1 + C2))
Rf_required = (R1_required - R_front_external_series) / N
Rt_required = R2_required / N  # only for verified N-series tail reduction

t_front_physics = k_front * sqrt((R1*C2)^2 + k_LC*L_effective*C2)
t_tail_physics = k_tail * R2 * (C1 + C2)
V_crest_physics = eta_expected * N * U_charge
```

If `D < 0`, the calculator target is below its inductance-only front limit. If `D = 0`, it requires zero front resistance, outside the positive resistor catalogue. Report the mode-specific condition; do not clamp into a buildable solution. This approximate inverse limitation is not proof that every alternative circuit is impossible.

Calculator mode omits basic capacitance only in the named workbook fixture. Its crest is a transfer-efficiency assumption. If charge is set from the target formula, crest equals target by construction. Its switching front formula is a workbook approximation, not a time-to-peak standard definition. These limitations remain visible; residuals do not turn an unverified circuit into a verified circuit.

5.3 Analytic baseline when L = 0

For the registered generator-side-tail capacitive circuit:

```
a = 1/(R1*C1) + 1/(R1*C2) + 1/(R2*C1)
b = 1/(R1*R2*C1*C2)
beta = (a + sqrt(a*a - 4*b))/2
alpha = b/beta
v2(t) = V0 * (exp(-alpha*t) - exp(-beta*t))
        / (R1*C2*(beta-alpha))
t_peak = log(beta/alpha)/(beta-alpha)
```

The characteristic equation is $s^2 + a*s + b = 0$, with roots `-alpha` and `-beta`. Use a stable `expm1`-based evaluation for close rates and a repeated-root limit when necessary. These are exact equations for the declared ideal circuit, not an exact model of all laboratory phenomena.

5.4 ODE baseline when L > 0

```
C1 * dv1/dt = -v1/R2 - i
L  * di/dt  = v1 - R1*i - v2
C2 * dv2/dt = i
initial state: v1=V0, i=0, v2=0
```

Here `L` is the registered circuit's `L_effective_H`; in the isolated workbook fixture it equals the supplied loop-inductance field. Use a stiff-capable integrator such as SciPy `solve_ivp` with `Radau`, scale states and use unit-appropriate error tolerances. A starting policy is relative tolerance `1e-9`, maximum step no larger than the expected front time divided by 200, and simulation horizon at least three target tail times. Adapt/extend until relevant crossings are resolved; a fixed horizon is not proof that a crossing does not exist. Store tolerances, status, step diagnostics, horizon and interpolation method. Check convergence by tighter tolerances/step settings on validation fixtures. SciPy integrator documentation.

Add other active branches and load models to a registered solver rather than inserting parameters into incompatible equations. For linear circuits, voltage/current scaling may be used to reduce repeated work, but keep the actual charge and rating checks explicit. Nonlinear temperature/voltage-dependent components invalidate unqualified linear scaling.

5.5 Waveform metrics

For a smooth full lightning impulse, compute crest and the ordered rising 30%/90% crossings, then:

```
t_front = 1.67 * (t90 - t30)
t_virtual_origin = t30 - 0.5 * (t90 - t30)
t_tail = t50_falling - t_virtual_origin
```

Use the direct virtual-origin expression above consistently. The formula book's `t30 - 0.3*T1` is a rounded equivalent, not exactly identical when 1.67 is used.

For the source legacy switching profile, use explicit time-to-peak and a declared origin for time-to-half-value. Do not deploy the book's unverified K-factor expression. Ringing, overshoot and chopped impulses need an appropriate standards-backed extraction implementation; return `extraction_unsupported` when absent. Keep `standard_compliance_verified=false` unless the selected definition and evaluator have been validated against the applicable standard. IEC's 2025 edition changes the switching front-time designation; it must be a separate profile. IEC edition record.

Overshoot limit 10% and waveform tolerances below are inherited project/formula-book policies, not numeric constraints found on the last PDF page. Record this provenance and validate applicability. A suitable waveform trace is required to assess overshoot; three corrected metrics alone cannot measure it.

5.6 Inverse refinement and fixtures

Calculator inverse values seed SMAC and optional coupled numeric refinement. In exact mode, do not assume time is globally monotone in resistance. Bracket positive-domain roots only after checking monotonicity/sign changes, or use bounded coupled optimisation with forward evaluations. Record nonconvergence. Never require an impossible exact round-trip for all loads/stage counts.

Standalone calculator fixture: benchmark profile, `N=9`, `C2=1.5e-9 F`, `L=18.5e-6 H`, charge `193089.43089430896 V`, required front `49.52213922066368 Ω`, required tail `23.942283773045347 Ω`. Actual front 50 Ω / tail 25 Ω gives front `1.210029958306818e-6 s`, tail `5.22088875e-5 s`, crest 1,425,000 V under the assumed efficiency.

A separate **L=0 ideal-circuit fixture**, with the same capacitances, actual resistors and charge, gives approximately front `1.9356218151e-6 s`, tail `5.6448212493e-5 s`, crest `1658013.7612 V`, efficiency `0.9540851117`. These values were calculated from the declared analytic model; they are not laboratory validation or the L=18.5 µH case. They illustrate why the residual model's baseline identity matters.

Acceptance: calculator round-trip for feasible inverse cases; benchmark numerical agreement; analytic initial/late-time conditions; analytic/ODE agreement as L approaches zero; energy balance for the stated circuit; waveform crossing ordering; zero/negative input handling; unavailable crossings and unsupported topology reported; crest responds to actual settings at fixed charge.

6. Work package C: DP networks, exact wiring and CP-SAT selection

Standalone inputs: two independent catalogues, required resistance pair, stage count, search/tolerance policy, candidate-specific precomputed feasibility metadata. **Outputs:** selected front/tail assemblies, alternatives searched, solver statuses and guarantee scope. **Interface:** `realise_pair(context, proposal, catalogue_options) -> RealisationResult`.

6.1 DP recurrence and stock states

For role-specific count vector `c`, store `(nominal_fraction, canonical_recipe, metadata)` for every retained assembly in the declared search scope. Vector length is the number of usable types in the supplied role catalogue, with each entry between zero and its `q_effective[type,N]`. Do not hardcode vector lengths of 10/12 or assume every type supplies four units. For every unordered split into nonempty sub-vectors `a+b=c`:

```
series(Ra, Rb) = Ra + Rb
parallel(Ra, Rb) = Ra*Rb/(Ra+Rb)
```

Build by increasing total part count. Represent nominal decimal values as exact rational numbers. Canonicalise commutative child order and equivalent associative groups. Keep a topology or feasibility frontier when equal equivalent resistances have different ratings/parasitics. The prototype's single witness per nominal value is insufficient for complete rating-aware selection.

Start with `max_parts=4` as a configurable computation budget. For the workbook example only, there are 1,000 nonempty front quantity vectors and 1,819 tail vectors in that region; full example stock spaces contain `5^10` and `5^12` vectors including empty. For arbitrary input, the full count-vector space is `product(q_effective_i + 1)` including empty. Count bounded nonempty states by summing coefficients for degrees 1 through K in `product(1+x+...+x^q_effective_i)`. Avoid promising exhaustive coverage of all available parts in an arbitrary input catalogue. Increasing the DP scope can discover a closer result; update the archive only after evaluating its complete operating point.

Nominal resistance tolerances are distinct from component manufacturing tolerances and waveform tolerances. Expose all three. A tolerance stop returns `satisfactory_within_scope`; exact completed bounded search may report `closest_nominal_within_scope`. Time-limited or pruned search reports `approximate` or `unresolved` with its scope.

6.2 Assembly JSON and connectivity

Equivalent resistance and global series/parallel counts are insufficient for arbitrary nested networks. Return a recursive tree **and** an explicit terminal netlist. Every leaf has a role-specific type and unique component occurrence ID. Example front assembly:

```
{
  "assembly_id": "front-example-4parts",
  "role": "front",
  "equivalent_ohm": 49.523809523809526,
  "nominal_fraction": {"numerator": 1040, "denominator": 21},
  "recipe": {
    "op": "parallel",
    "children": [
      {"op": "series", "children": [
        {"op": "resistor", "component_id": "F100-1", "type_id": "front_100", "ohm": 100},
```

```
          {"op": "resistor", "component_id": "F30-1", "type_id": "front_30", "ohm": 30}
        ]},
        {"op": "series", "children": [
          {"op": "resistor", "component_id": "F50-1", "type_id": "front_50", "ohm": 50},
          {"op": "resistor", "component_id": "F30-2", "type_id": "front_30", "ohm": 30}
        ]}
      ]
    },
    "netlist": [
      {"component_id": "F100-1", "node_a": "in", "node_b": "branch_a"},
      {"component_id": "F30-1", "node_a": "branch_a", "node_b": "out"},
      {"component_id": "F50-1", "node_a": "in", "node_b": "branch_b"},
      {"component_id": "F30-2", "node_a": "branch_b", "node_b": "out"}
    ],
    "type_counts": {"front_100": 1, "front_50": 1, "front_30": 2},
    "total_resistors": 4,
    "top_level_parallel_branches": 2,
    "branch_summary": [
      {"branch": "a", "series_resistors": 2, "equivalent_ohm": 130},
      {"branch": "b", "series_resistors": 2, "equivalent_ohm": 80}
    ],
    "ratings_status": "unknown"
  }
```


Provide a readable expression `(100 + 30) parallel (50 + 30)`, canonical topology hash, inventory usage and exact connections. For nested networks, return summaries per node rather than inventing a single “parallel count”. Expand occurrence IDs by stage, e.g. `stage_03/front/F30-2`, and expose every resistor's stress record.

6.3 CP-SAT's finite-option model

DP generates finite assembly candidates. CP-SAT chooses among them; it does not directly solve the nonlinear ODE, infer thermal ratings or enumerate arbitrary networks from three scalar variables. Its constraints use integer representations. OR-Tools CP-SAT documentation.

For options `f` and `t`, introduce Boolean selectors `x_f` and `y_t`:

```
sum(x_f) = 1
sum(y_t) = 1
sum(x_f * count_f[type]) <= q_effective_front[type,N]
sum(y_t * count_t[type]) <= q_effective_tail[type,N]
```

`N` is the SMAC-proposed supported integer and is validated here; CP-SAT must not secretly substitute another stage count. Approved recipes filter options. Forbidden front/tail combinations use pair exclusions. Global role-specific stock, if available, additionally imposes `N*selected_count <= global_available`. No combined front/tail bound is added.

Precompute integer-scaled resistance mismatch, quantities and verified candidate-dependent rating/compatibility flags. Use conservative integer conversions at hard-bound edges and recheck original exact/floating values after selection. For explicit mismatch tolerance, filter exact fractions before integer optimisation. Optimise mismatch first and parts second with sequential objectives or a provably dominating integer coefficient. Record scaling resolution, incumbent cost/bound, selected IDs, status and time budget.

Statuses `OPTIMAL`, `FEASIBLE`, `INFEASIBLE`, `UNKNOWN`, `MODEL_INVALID` are exposed distinctly. `FEASIBLE` does not certify the best option; `UNKNOWN` does not prove infeasibility. A finite pool's `INFEASIBLE` means no solution in that pool, not no possible resistor network.

Ratings may depend on charge, stage count, load and the chosen front/tail pair. Do not cache a rating-valid label across incompatible operating points. Select, simulate, reject an invalid pair with a no-good constraint, and retry within a deterministic inner budget; retain diagnostic records. For small independent pools, direct enumeration is a valid reference implementation to validate CP-SAT.

6.4 Tolerance and thermal extensions

For independent resistor tolerances, equivalent resistance is monotone in each leaf resistance. Nominal/low/high equivalent values can be computed recursively using all-low and all-high leaves, giving a static resistance interval. This does not by itself bound coupled waveform or per-leaf stress; use validated scenario evaluation for those.

Fixtures: under a supplied four-unit profile, four front 30 Ω and four tail 30 Ω accepted and five in either role rejected; a two-unit type rejects three uses; an input with three front and five tail types and non-workbook values works; global quantity 10 with N=4 permits two units per stage; the four-leaf network above has exact resistance `1040/21`; permitted/forbidden recipe pairs; a CP timeout without proof; a 6,000 Ω front request outside the workbook example bound but potentially feasible under a different supplied catalogue; different equivalent networks retaining rating metadata.

Acceptance: arbitrary supplied catalogue sizes/values accepted; quantity-dependent state vectors and bounds; inventory hashes isolate DP caches; reconstructible connected two-terminal netlist; exact tree/netlist equivalence; no reused physical leaf; no cross-role stock coupling;

exhaustive bounded tiny-catalogue comparison; integer-boundary recheck; no best/global-feasibility overclaim; recipe unchanged through serialisation.

7. Work package D: Component stress, heat and residual ML explanations

Standalone inputs: context, realised assembly pair, baseline trace or mock trace, measured-data/feature manifest and residual model adapter. **Outputs:** electrical/thermal stress records plus residual correction, support report and per-output explanations. Keep stress physics separate from statistical attribution.

7.1 Per-resistor electrical and thermal calculation

For the base uniform-stage ideal model, each repeated stage front assembly sees `Rf_actual*i(t)`; the combined internal front assemblies see `N*Rf_actual*i(t)`. An external series front resistor sees its own `R_external*i(t)` and is reported separately; do not divide the full R1 voltage, including that external resistor, equally across internal stage assemblies. Total tail-network voltage is `v1(t)`; under the same ideal equal-sharing assumption, each stage tail assembly sees `v1(t)/N`. Label this assumption. A single equivalent capacitor model cannot establish unequal stage/transient stresses; a detailed stage circuit or verified envelope is needed where applicable.

Given a node's voltage/current and its subtree equivalent:

- Series children carry the same current; each child's voltage follows its resistance.
- Parallel children have the same voltage; each child's current follows its conductance.
- Recurse to every leaf and compute `p_j(t)=i_j(t)^2 R_j=v_j(t)^2/R_j`.
- Integrate `E_j = integral(p_j(t) dt)` over a resolved waveform; record integration horizon and estimated remaining energy.
- Report peak absolute voltage/current/power, pulse energy, resistor tolerance, rating margins, stage index and source trace.

For repetitive shots, average power estimate is `E_j * shots_per_second`, with the source maximum `2/60 s^-1`. This is not the peak pulse power. With known thermal resistance `R_theta_K_per_W` and heat capacity `C_theta_J_per_K`, a simple lumped model is:

```
C_theta * dT/dt = p(t) - (T - T_ambient)/R_theta
adiabatic single-pulse rise estimate: delta_T = E_pulse / C_theta
```

Use a validated thermal model and initial thermal state for repeated heating. Without thermal constants or a temperature sensor, return `temperature_estimate_K=null`, `thermal_status=unknown`. Do not translate pulse energy into a fabricated temperature. Preserve charging versus discharge branch phases; charging resistor heat is not identical to impulse front-resistor heat.

Stress/rating checks remain hardware constraints and must not be delegated to ML. If temperature-dependent resistance is implemented, declare an electrothermal solve/update policy and physics version; the constant-R formulas no longer capture that feedback automatically.

7.2 Residual-only ML contract

Train one residual regressor per output and impulse category initially: six regressors for the two supplied categories, with three selected for a particular run. This keeps the different timing scales and definitions explicit.

```
y_front = measured_front_s - physics_front_s
y_tail  = measured_tail_s  - physics_tail_s
y_crest = measured_crest_V - physics_crest_V

residual = model.predict(validated_features)
corrected_front_s = physics_front_s + residual_front_s
corrected_tail_s  = physics_tail_s  + residual_tail_s
corrected_crest_V = physics_crest_V + residual_crest_V
```

Pin TabPFN **v2** explicitly and record checkpoint/package versions; do not rely on a library's changing default model. The official implementation exposes model-version selection. TabPFN implementation.

The runtime feature manifest defines ordered names, units, transformations, required/optional status, training ranges and provenance. Candidate fields include realised `N`, charge, actual Rf/Rt, registered load/setup, baseline outputs, stage capacitance and properly recorded environment. Per-assembly/pulse-energy/thermal features may be added only when consistent training measurements/baselines are available. Expected efficiency may be a known request feature; measured post-shot efficiency is not available at prediction time.

Feature preparation is a standalone adapter: select only fields listed in the manifest; remove constant features within the category; fit imputation/encoding and any scaling on training records only; preserve missingness indicators when justified; and apply the identical ordered transform at inference. Keep realised settings and charging voltage in the contract even if a constant setting is omitted from a category's fitted matrix. Perform feature selection using the training/validation split, never the hidden test. Record each retained/excluded feature and its reason. A selected model must be tested against the simple residual baseline before acceptance; small sample size is not a reason to omit validation.

Initial support gate: reject unknown impulse/load/topology categories and unavailable required features; check each continuous feature against the recorded per-category training support envelope, including constant fields; and compare the baseline/feature versions exactly. Thus a lightning tail setting other than 25 Ω is outside the current workbook's demonstrated tail support. In-range individual features are only a necessary support check, not proof that their joint combination was validated. Add held-out object/setup validation and a versioned joint-support criterion before broadening a model's accepted domain. Store `support_status`, violated feature names, training envelopes and reason codes. Do not silently extrapolate to a new transformer model, new environment or incompatible physics profile. A new inventory hash alone does not mandate rejection when all model-relevant realised values/features remain supported, but changes in component ratings, assembly effects and represented thermal/parasitic features must be assessed. Inventory acceptance is distinct from prediction-domain support.

Do not include observed outputs, residual labels, split/row IDs or unavailable future measurements as inputs. Do not use the desired target as a proxy for achieved voltage. If target values are retained as request context or in a benchmark feature set, document their availability and audit target-to-baseline leakage.

The model predicts at fixed actual settings. It cannot modify `N`, charge or assemblies. Reject or flag nonpositive corrected times/crest as an invalid prediction; do not silently clip and call it valid. Changed physics topology, fidelity or extraction definitions require regenerated residual labels and revalidation. Existing workbook residuals are valid only for their original calculator baseline.

7.2.1 Explicit training targets and delivered outputs

There are three supervised training targets per category, all continuous residuals. The model does not train on requested 1.2/50 targets as labels and does not train to choose N/Rf/Rt.

Training label in SI	Definition	Existing workbook residual column	Delivered corrected output
`residual_front_s`	Observed front/peak minus matching physics front/peak	`Residual_FrontPeak` (µs, convert by `1e−6`)	`physics_front_s + residual_front_s`
`residual_tail_s`	Observed half-value time minus matching physics tail	`Residual_Tail` (µs, convert by `1e−6`)	`physics_tail_s + residual_tail_s`
`residual_crest_V`	Observed crest minus matching physics crest	`Residual_Crest` (kV, convert by `1000`)	`physics_crest_V + residual_crest_V`

The observed columns are `Observed_FrontPeak_us`, `Observed_Tail_us`, `Observed_Crest_kV`; corresponding baseline columns are `Physics_FrontPeak_us`, `Physics_Tail_us`, `Physics_Crest_kV`. Recompute and check residual identities after SI conversion. The workbook's observed columns are synthetic observations, not actual measured shots. For lightning the first output is conventional front time; for legacy switching it is time-to-peak. Keep these definitions with category-specific models.

Requested front/tail/crest values are optimiser goals. Observed outputs/residual labels exist during supervised fitting and evaluation only, never as inference features. The current model predicts three scalar corrections; waveform samples remain the explicitly labelled reconstructions in Section 10.1.

7.2.2 Screenshot-aligned ML inputs after extraction

Screenshot input group	API/feature fields	Current workbook availability
Candidate generator settings	`N`, `U_charge_V`, actual per-stage resistances, exact front/tail recipes and generator connection	`Stages`, `Charge_kV_Stage`, `Front_R_Stage`, `Tail_R_Stage` available; exact wiring/topology not recorded
Load and circuit characteristics	Object/divider/stray/basic capacitances, effective generator/load capacitances, effective front/tail resistance and loop inductance	Load/divider/stray and loop L recorded; equivalents derivable under the declared benchmark topology; basic C uses the profile inclusion policy
Impulse information	Category and waveform-definition ID	`Impulse_Type` available; category selects one of the two model groups
Physics baseline predictions	Front/peak, tail, crest and overshoot with baseline version	Three scalar baseline columns available; overshoot absent and cannot be reconstructed from those three alone
Additional project conditions	Environment, component/thermal summaries, supported general-load model/terminals	Absent from current workbook; require additional evidence and a new validated feature manifest

Convert stored units to SI, derive equivalents with Section 5.1.1, then apply the fixed training-fitted transformation. A recipe tree is retained as structured evidence; a tabular regressor needs a declared encoding, such as role-specific part counts, branch/depth summaries and validated stress/parasitic features. Raw JSON trees must not be passed to a numeric model without an encoder. Do not manufacture exact wiring from an equivalent scalar resistance in historical rows.

In the **initial workbook model**, use recorded settings/load/setup and the three scalar physics baseline values, with justified derived features. Omit unsupported recipe, overshoot, environment and thermal features from that fitted matrix while returning their explicit unavailable statuses in the

API. In a later full-feature model, enable them only when consistent training and inference data exist. Constants may be retained in the API and excluded from the model matrix; an unavailable field is not the same as a constant zero.

`Efficiency` is a recorded expected estimate, not a per-stage measured label. `Test_kV` is known request context but the workbook baseline crest equals it by construction; document this dependency and test the correction model without relying on target voltage as a shortcut. IDs and split columns are excluded from features. Redundant derived inputs are assessed using training/validation performance rather than assumed to add independent information.

The screenshots therefore define the desired feature contract, **not evidence that feature extraction has already been executed or that every field exists in this workbook**. Publish a manifest with each feature's `available`, `derived`, `unavailable`, `selected`, `excluded_constant` or `excluded_after_validation` status. A module-level extraction fixture must reproduce all recorded/derivable fields and preserve unavailable-field reasons.

7.3 Dataset scope and standalone training

The workbook contains 2,000 synthetic records: 1,000 per impulse category; each has 700 training, 150 validation and 150 hidden-test records. Lightning tail resistance is constant at 25 Ω, switching tail is 1,195 or 1,200 Ω, stage ranges are 8–10 and 6–7 respectively. Environmental measurements, general load R/L and resistor temperatures are absent. These facts restrict supported predictions and optimisation, regardless of model family.

A data module can independently read the workbook and reproduce calculator-baseline residual labels. For a different physics baseline, use a supplied baseline fixture/adapter to regenerate labels. Real measured shots are needed to validate physical corrections. Report synthetic-only models as `diagnostic`, not laboratory-validated. Store per-category absolute errors, normalised errors, bias and predicted-acceptance/measured-failure rates; keep hidden test untouched until selection is complete.

Compare physics-only, a simple mean/linear residual baseline and residual TabPFN v2. A trained TabM challenger may be added later behind the same residual contract; it does not change this specification's residual-only decision. Split actual lab data by object/setup/session where appropriate. Freeze feature preparation, support rules and model versions before an optimisation run.

A model may provide calibrated prediction intervals only after appropriate held-out validation. Do not label ensemble spread or synthetic test error as guaranteed uncertainty for unseen transformer/environment conditions.

7.3.1 Epochs, fitting and validation policy for 700 records

The verified split is **700 training / 150 validation / 150 hidden-test records per category**. Fit all preprocessing and each category's residual regressors using only its 700 training rows. Use validation for feature/model selection and hidden test only for the final frozen-model assessment. Do not combine the hidden test with training while claiming the same held-out result.

For the selected **standard pretrained TabPFN v2**, downstream fitting uses in-context examples rather than gradient training through repeated epochs. Set `training_mode=pretrained_in_context`, `epochs=null`, `epochs_applicable=false`, and `downstream_gradient_updates=0`. Supply each output regressor its 700 feature/label examples, then evaluate predictions on validation/test. Fitting/context preparation and inference still require computation; zero downstream updates does not mean zero runtime or perfect accuracy. Model pretraining has already occurred separately. Prior Labs model description, versioned implementation.

There is no theoretical epoch-to-convergence number determined by 700 rows. Even a correctly implemented gradient-trained model's convergence depends on residual complexity, noise, feature scaling, architecture, learning rate and regularisation. More epochs cannot supply missing environmental or resistance variation. SMAC candidate iterations are not ML training epochs.

If a **Residual TabM challenger** is later trained, use this explicit initial experiment budget, not a promised convergence estimate:

- Maximum 200 epochs, validation every epoch, early-stopping patience 20 epochs.
- Batch size 32 gives `ceil(700/32)=22` optimiser updates per epoch when retaining the last partial batch, at most 4,400 updates per category in this budget.
- Standardise residual targets using training-only statistics or normalise losses by declared positive physical scales; never add raw time and voltage losses.
- Track mean validation absolute error in tolerance-normalised corrected outputs, each output's separate error, training loss and acceptance/failure rates. Use a fixed validation criterion to select the checkpoint.
- Save the best validation checkpoint and restore it; stop after 20 epochs without improvement in that criterion. Define improvement as a relative decrease of at least `1e−4` from the current best positive score, with an explicitly handled zero-score case. This is a proposed experiment policy, not an equipment constraint or an official TabM default.
- Review progress at epochs 10, 25, 50, 100 and 200. Report the selected epoch and actual stopping epoch; do not report the maximum budget as a measured convergence time.
- Use fixed seeds, preferably three declared runs if compute permits, to assess variability. Hyperparameter choices are selected with validation, never hidden-test performance.

TabM's reference implementation supports validation-driven early stopping. Official TabM implementation, reference training loop. The batch/epoch/patience numbers above are this project's starting proposal and require experimental assessment. TabPFN fine-tuning, if deliberately introduced later, is a separate versioned training mode with its own protocol; it is not part of the current standard-fit decision.

Expose a `ModelFitReport`: category; target names/units; train/validation/test counts; feature/baseline hashes; model/checkpoint versions; fitting mode; epochs applicable; maximum/selected/stopped epochs or `null`; gradient update count; validation criterion/history; seeds; per-output final metrics; and synthetic-versus-measured provenance. No model fitting benchmark or actual convergence result is claimed by this specification.

7.4 Explain each correction honestly

Return an explanation for each residual and corrected output. Use a model-compatible, versioned feature-attribution method; a model-agnostic SHAP approach is one option. Store background-set identity, feature group policy, seed, runtime and additivity error. SHAP explainer documentation.

For an additive attribution with identity link:

```
residual_prediction ≈ reference_residual + sum(feature_contribution)
corrected_prediction = physics_baseline + residual_prediction
```

Feature contributions have the residual output's units. Explain whether increased temperature, pulse-energy or humidity features contributed positively or negatively relative to the selected reference, **only when those features were actually used and supported**. Correlated and derived features complicate attribution; preserve grouping and do not claim causal proof.

Per-resistor heat is calculated evidence from Section 7.1. It is not automatically an explanation of the ML prediction. If the model uses only aggregate pulse-energy features, link the aggregation to per-resistor records and state that no unique per-resistor ML attribution exists. If thermal/environment fields are absent, explanation must explicitly say “not modelled by this version”. Never generate “heat caused this correction” merely because resistors dissipate energy.

For all candidate evaluations retain model inputs, baseline, residuals and support checks. Full per-output attributions are required for final recommendations and available through the explanation API for other evaluations; expensive explanation work is cached and has a declared budget/status. Never change predictions while generating explanations.

Fixture: baseline `[1.21e−6, 52.2e−6, 1425000]`; mock residual `[−0.01e−6, −2.2e−6, −5000]` gives `[1.2e−6, 50e−6, 1420000]`. Label mock attribution values; a missing-thermal-data fixture returns no temperature claim. A 100 Ω resistor carrying a constant 2 A for 1 ms dissipates 400 W and 0.4 J; with a declared mock `C_theta=0.2 J/K`, its adiabatic rise is 2 K.

Acceptance: exact residual addition; frozen model settings; correct baseline-version rejection; no label leakage; unsupported points flagged; per-leaf energy reconciles with assembly energy; unknown thermal data stays unknown; attribution additivity within declared numerical error; no explanation of a feature absent from the model.

8. Work package E: Objective, SMAC3 and the top-three archive

Standalone inputs: context plus interchangeable evaluator protocol `evaluate(proposal)->CandidateEvaluation`. **Outputs:** full run history, final top-three and stop/shortfall reports. Implement and test first with a deterministic mock evaluator; physics, DP and ML need not exist yet.

8.1 Error and tolerance definitions

The formula book contributes worst tolerance-normalised error; the working architecture contributes explicit absolute deviations, a weighted combination and the three-result archive.

For outputs `p` in front, tail, crest:

```
signed_delta_p = corrected_p − target_p
absolute_delta_p = abs(signed_delta_p)
percentage_delta_p = 100 ∗ signed_delta_p / target_p
allowed_delta_p = positive_relative_tolerance_p ∗ target_p
normalised_error_p = absolute_delta_p / allowed_delta_p
S = max(normalised_error_front, normalised_error_tail, normalised_error_crest)
J = sum(weight_p ∗ normalised_error_p) / sum(weight_p)
waveform_within_tolerance = (S <= 1)
```

Do not add raw seconds and volts. `S` prevents a good average hiding one failing output. `J` compares total normalised deviation when the worst error ties. Weights default to equal positive values; record any chosen change.

Versioned source/formula-book tolerance profiles:

Profile	Front/peak	Tail	Crest	Basis
`lightning_project_v1`	1.2 µs ±30%	50 µs ±20%	Requested crest ±3%	Supplied formula book/project test policy

Profile	Front/peak	Tail	Crest	Basis
`switching_workbook_legacy_v1`	Peak 250 µs ±20%	2500 µs ±20%	Requested crest ±3%	Workbook comparison policy
`switching_legacy_extended_tail_v1`	Peak 250 µs ±20%	2500 µs ±60%	Requested crest ±3%	Formula-book legacy policy requiring standards verification

These tolerances are not taken from the PDF's last-page hardware table. Store exact profile, provenance and verification status. Do not equate a project tolerance verdict with verified standards compliance. Declared absolute tolerances may be supported as a separate schema; their scales must stay positive.

8.2 Search scalar and archive ordering

Default SMAC minimises a finite scalar prioritising the worst output:

```
For hardware-feasible, ML-supported, valid predictions:
    cost = S / (1 + S)                  # in [0,1)
For hardware-failed/unknown, unsupported, DP-unresolved or invalid predictions:
    cost = 2                           # infeasible/unevaluable penalty
```

This bounded monotone transform preserves `S` ordering and ensures a hardware violation cannot appear preferable to an eligible candidate. Keep individual failure codes and uncertainty; penalty cost is not a predicted waveform. The archive uses lexicographic ordering:

```
(S, J, total_parts_per_stage, N, U_charge/U_stage_max, canonical_candidate_id)
```

Only eligible records enter the archive. Since `S<=1` precedes `S>1`, satisfactory candidates automatically outrank outside-tolerance candidates. Parts/stages/utilisation break actual waveform-objective ties; they do not override a waveform improvement. Compute objective values at declared precision and avoid undocumented “nearly equal” comparisons. If an epsilon policy is selected, version and expose it.

The formula book suggests `S + .002*parts + .001*N + .01*utilisation`. Do not use these additive weights as guaranteed tie-breakers: with many parts they can outweigh waveform improvements. An optional user-selected weighted multi-objective policy must explicitly accept that tradeoff. The primary objective still uses corrected predictions, not only the ideal physics targets.

Resistance mismatch is the inner assembly-selection objective. It is not the outer waveform objective. A closer resistor does not necessarily give a better corrected waveform.

8.3 SMAC3 adapter

SMAC3 proposes configurations and receives evaluated costs. Configure a bounded space with integer `N` and positive, preferably log-scaled required front/tail resistances inside declared searchable intervals. Derive intervals from catalogue/physics bounds, never from synthetic training coverage alone; separately reject unsupported ML points. Seed with feasible inverse estimates and nearby alternative stage counts.

Use `HyperparameterOptimizationFacade` or a pinned appropriate facade with ask/tell; the evaluator returns the scalar above. Set deterministic behaviour only when repeated context/candidate evaluations are reproducible. Persist seed, configuration space, facade settings, trial records and run context hash. SMAC3 getting started, ask/tell API.

SMAC's surrogate is an internal search model, separate from the residual correction ML engine. Updating SMAC from candidate costs does not retrain the correction model or convert predicted outputs into laboratory labels.

Different continuous resistance proposals may snap to one actual setup. Canonical identity includes `N`, actual topology/part IDs, charge, hardware/load/setup/model versions. Cache the complete evaluation, return its cost to the corresponding trial and increment proposal counts even for duplicates. Log both proposal and realised identities.

For several permitted DP alternatives at one proposal, define a fixed policy. Initial implementation evaluates one deterministic selected pair per trial; CP-SAT rating retries are bounded and counted. A later multi-pair policy must charge all forward/ML evaluations to their budgets and declare how one trial cost is aggregated. Never let extra internal evaluations bypass the run cap.

8.4 Top-three archive and stopping

Retain the three best **distinct complete realised eligible setups** across all evaluations. Store earlier good results even when later proposals are worse. Do not mix one candidate's front assembly with another's tail assembly. Requested values snapping to the same realised setup do not fill multiple positions. Canonical equivalent redrawings are one entry.

Equal equivalent resistances with different approved physical wiring may be distinct alternatives, but if the ML features cannot distinguish their parasitics/stress, do not imply different predicted outcomes. Expose this explanation limitation.

Required deterministic budgets: maximum proposals, forward/ML evaluations, DP states/time/part count, CP-SAT time/retries, repeated-proposal window and archive-stagnation window. Wall time is an additional operational cap; counts are the reproducible cap. Track improvement in all three archive slots, not only the leading S.

Stop when three distinct eligible setups meet all waveform bounds, the declared stagnation condition occurs, a declared count/time budget expires or no supported feasible evaluation can be performed. Finding one predicted exact match alone does not satisfy the three-setup goal. Successful threshold stopping does not establish the global top three.

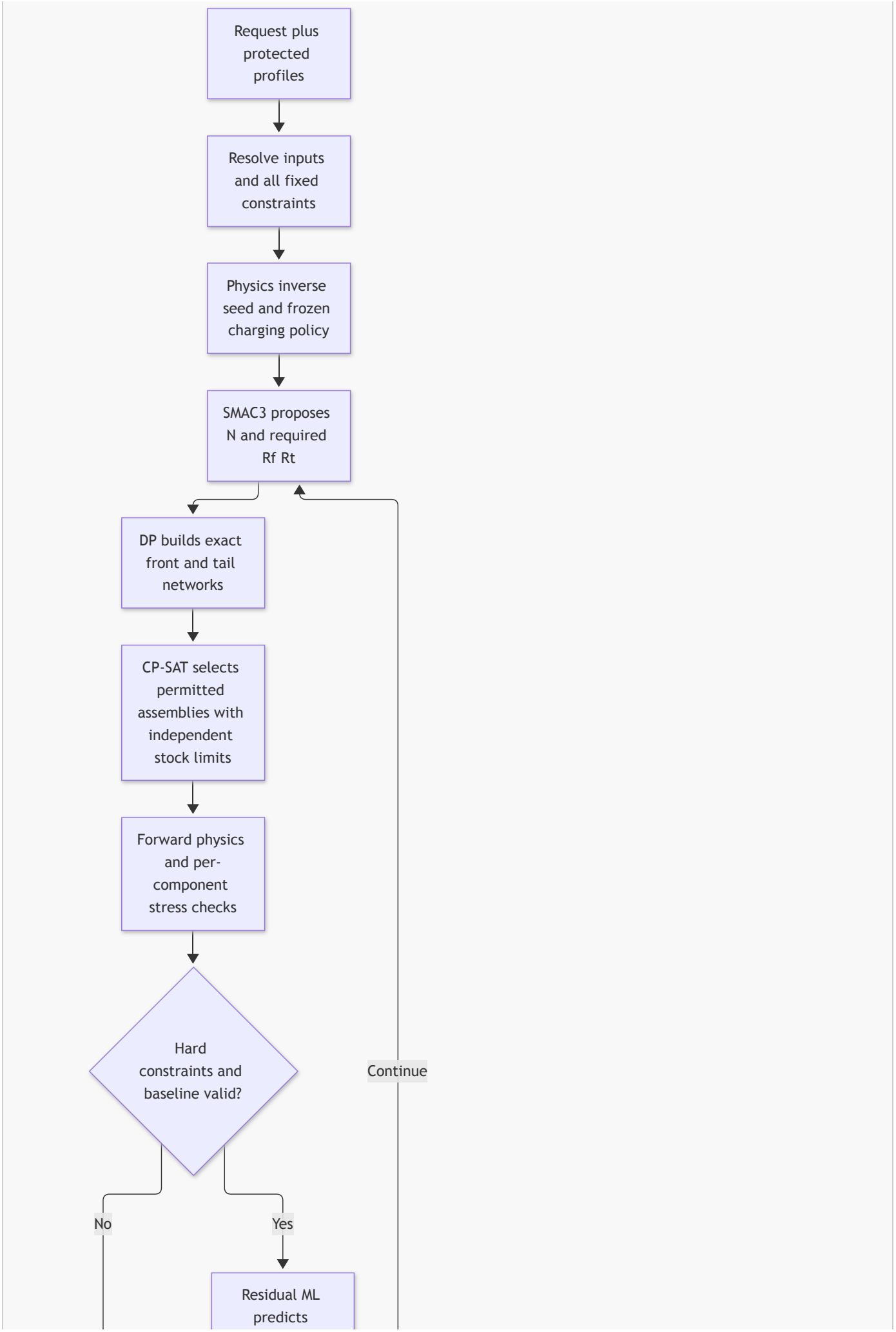
If fewer than three satisfactory setups exist in the evaluated history, return up to three eligible entries with individual outside-tolerance flags as necessary. If only one or two eligible entries exist, return them and an explicit shortfall reason. Return an empty list with rejection counts if none exist. Never include hardware-failed/unknown or unsupported candidates as verified alternatives.

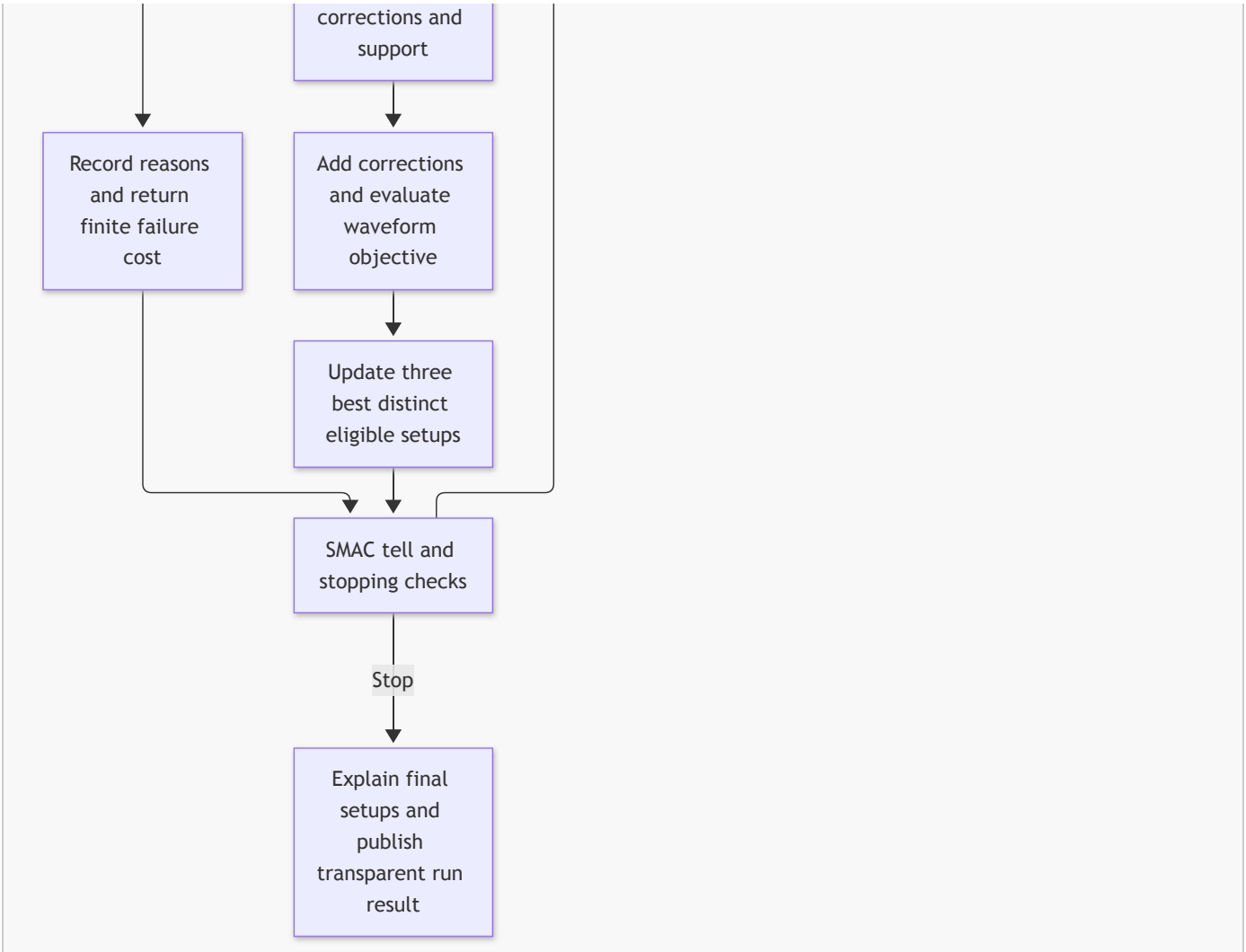
Fixtures: evaluator where required values snap to duplicates; ten candidates with known scores; later worse result; one exact result followed by two distinct satisfactory ones; no eligible candidates; second/third-slot improvement while leader is unchanged; finite failure cost; count exhaustion despite invalid proposals.

Acceptance: ordering matches the documented objective; top-three identity uniqueness; no hardware penalty entering archive; proposals change only declared controls; all inner evaluations counted; same seed/context reproducible where supported; stop reasons and search scope exposed; no global optimum guarantee.

9. Work package F: Pipeline orchestration and stage trace

Standalone inputs: context and five protocol adapters: inverse physics, assembly realisation, forward/stress physics, residual correction, SMAC adapter. **Outputs:** candidate records, ordered events and final run result. Implement using mocks for every adapter.


Request plus protected profiles
Resolve inputs and all fixed constraints
Physics inverse seed and frozen charging policy
SMAC3 proposes N and required Rf Rt
DP builds exact front and tail networks
CP-SAT selects permitted assemblies with independent stock limits
Forward physics and per-component stress checks
Hard constraints and baseline valid?
Continue
No
Yes
Residual ML predicts



Conceptual pseudocode:

```
context = resolve_and_freeze(request)
seed = physics.initial_proposals(context)
context = establish_and_freeze_charge_policy(context, seed)
smac = initialise_smac(context, seed)
archive = empty; history = empty; cache = empty

while proposal_budget_remains and evaluation_budget_remains:
    trial = smac.ask()
    proposal = proposal_from_trial(trial)
    count_proposal()
    realised = assembly.realise_pair(context, proposal)
    if realisation failed or unresolved:
        record(stage_trace, constraints, failure_cost=2)
        smac.tell(trial, cost=2)
        apply_stopping_rules()
        continue
    assign_declared_charge(realised, context)
    if canonical_identity(realised, context) in cache:
        evaluation = cache[identity]
        record_cache_hit_and_proposal_link()
    else:
        physics_result = physics.evaluate_actual(context, realised)
        count_forward_evaluation()
        if hard_checks_failed_or_unknown(physics_result):
            evaluation = rejected_evaluation(physics_result, cost=2)
        else:
            correction = residual_model.correct(context, realised, physics_result)
            count_ml_evaluation_if_invoked()
            evaluation = objective_and_eligibility(physics_result, correction)
        cache[identity] = evaluation
    append_history(proposal, evaluation)
```

```
    if evaluation.eligible:
        archive.insert_deduplicate_and_keep_three(evaluation)
    smac.tell(trial, evaluation.cost)
    if three_satisfactory_distinct(archive) or other_stop_rule:
        break

explain_each_returned_candidate(archive, context)
return full_result(archive, history, limits, stop_reason, shortfall)
```

Run stage records contain input record references/hash, output record references, resolved constraints, timings, cache status, errors and model/solver versions. Expose both algorithmic stages and physical generator stages. A stage 4 API trace means pipeline stage unless clearly labelled `generator_stage_index`.

Cancellation stops scheduling new proposals, preserves history and returns the best archive already evaluated. Resume only with the same frozen context and compatible pinned adapters. Changed physics/model/profile requires a new run or explicit full reevaluation, not reuse of incompatible scores.

Acceptance: each adapter independently mocked; failure short-circuit visible; no ML call for hardware-failed candidate; all input/output links retained; resume/cancel bounded; actual settings, baseline and residual always refer to the same candidate.

10. Work package G: Explanation and recommendation serialisation

Standalone inputs: a supplied `CandidateEvaluation` fixture and profile registry. **Outputs:** a complete readable/JSON recommendation; no frontend dependency.

Every recommended setup exposes:

1. Proposed `N`, required resistances and actual realised resistances.
2. Charge policy/value, generator stage connection and per-stage capacitance/energy.
3. Exact front/tail topology trees, netlists, component IDs and quantities; repeated-stage expansion.
4. Every resistor's available ratings, peak stress, pulse energy and thermal estimate/status.
5. Physics baseline, signed ML residual, corrected result and target, separately for front/tail/crest.
6. Absolute/relative/normalised errors, `S`, `J`, SMAC cost, archive order and tie-break fields.
7. Per-output feature contributions/reference, model inputs, support limitations and explanation status.
8. All fixed and requested constraints, observed values, margins, pass/fail/unknown and source.
9. Predicted tolerance verdict, hardware verification status, lab validation status and stop/shortfall reason.
10. Full context/model/physics/inventory hashes and iteration/search scope.

The result must include numeric waveform data for **one target reference and one curve for each of the three returned candidates**. The current ML model corrects front, tail and crest; it does not predict every voltage sample. Therefore candidate plots use a declared reconstruction from those ML-corrected parameters, labelled `ml_parameter_reconstruction`. Keep physics and measured traces separately labelled. Section 10.1 defines construction, sample fields and validation. Recommendations are alternatives for the same scenario, not three assemblies to build simultaneously.

Fixture: use the four-leaf front tree from Section 6 with mock tail and residual records. Serialisation must preserve topology and clearly show unknown ratings. **Acceptance:** lossless round-trip, no dropping unavailable fields, no measured-label substitution, all three candidates use separate complete records.

10.1 Work package H: Target and candidate waveform samples

Standalone inputs: waveform profile and target metrics, plus up to three candidate IDs with their ML-corrected front/peak time, tail time and crest. No complete physics engine or ML implementation is needed; metric fixtures suffice. **Outputs:** `WaveformPlotBundle` containing numeric time/voltage values, fit metadata, annotations and status for every curve. **Interface:** `build_waveform_plot_bundle(context, candidate_metrics, sampling_policy)`.

Required plots and provenance

Provide data supporting:

1. One comparison overlay: the target reference plus candidate 1, candidate 2 and candidate 3, with stable candidate IDs/ranks and distinct labels.
2. Three individual comparison plots: target versus each candidate, using the same target definition and comparable axes.
3. A front/peak zoom and a full tail view, so the front is visible despite the longer tail scale.

The backend supplies plot-ready samples and metadata; frontend layout remains deferred. If fewer than three candidates are returned, include only those curves and the same shortfall reason. Preserve a target-only plot when no eligible candidates exist. Do not fabricate candidates to complete a four-curve overlay.

Use curve origins `target_reference`, `ml_parameter_reconstruction`, `physics`, or `measured`. Legends must say, for example, **"Candidate 1 — reconstruction from ML-corrected parameters"**. The target is a representative reference shape for the requested metrics, not the unique waveform implied by three numbers. Neither a fitted reference nor an ML-parameter reconstruction is a measured laboratory trace, a pointwise ML prediction or evidence of predicted ringing/overshoot.

Default reconstruction: normalised double exponential

Use a versioned double-exponential family for smooth full lightning and supported legacy switching reference curves:

```
v(t) = V_crest * [exp(-alpha*t) - exp(-beta*t)] / q_peak, t >= 0
q_peak = exp(-alpha*t_peak) - exp(-beta*t_peak)
t_peak = log(beta/alpha) / (beta-alpha)
0 < alpha < beta
```

This is a shape fit, independent of the candidate's circuit-component equations. Do not reinterpret fitted `alpha`/`beta` as new physical resistor settings.

Fit the curve separately for the target and every returned candidate. For lightning, match the declared virtual-origin front and tail definitions; for the source legacy switching profile, match time-to-peak and time-to-half-value from the declared origin. A different switching-front definition needs its own registered fitter/extractor.

An independently implementable dimensionless fitting method is:

```
r = alpha/beta, with 0 < r < 1
tau = beta*t
q_r(tau) = exp(-r*tau) - exp(-tau)
tau_peak = -log(r)/(1-r)
w_r(tau) = q_r(tau) / q_r(tau_peak)

Extract dimensionless front_or_peak(r), tail(r) with the selected profile.
Find a bracket and solve:
    front_or_peak(r) / tail(r) = desired_front_or_peak_s / desired_tail_s
beta = front_or_peak(r) / desired_front_or_peak_s
alpha = r*beta
Scale peak voltage to desired_crest_V.
```

Scan/bracket before root finding; do not assume a feasible ratio or global monotonicity for every registered profile. Use stable exponential evaluation, root tolerances and analytic peak calculation. After fitting, re-extract metrics using the same profile and require relative error no greater than `reconstruction_metric_rtol` (default `1e-4` for this plotting policy). Store desired/re-extracted metrics and errors. This tolerance is a reconstruction-quality policy, separate from waveform acceptance tolerances.

A nonpositive metric, impossible ratio, missing root or extraction failure yields `reconstruction_status=failed` with a reason, not a fabricated curve. Candidate scalar predictions and their original eligibility remain visible. Mark the plot bundle `partial` if any required curve cannot be reconstructed; never hide the plotting failure. Ringing/chopped profiles unsupported by this family require a registered alternative or an explicit unavailable plot status.

Common grid, horizons and annotations

Generate one strictly increasing SI `time_s` array for all fitted curves. Default origin is the start of the reconstructed impulse, not virtual origin or peak. Record virtual origins and other timing landmarks separately; do not shift each candidate to its peak and hide timing differences. If a different alignment is offered, declare the transform and expose unshifted samples.

Use an adaptive/common union grid dense enough around all rising crossings and peaks, with a tail region resolving all falling-half crossings. Start with a front spacing no larger than the smallest front/peak target/prediction divided by 200. Include all curve landmarks explicitly. Default horizon is at least three times the largest relevant tail value and extends until every curve falls below 1% of its crest; enforce a declared point/time cap and report truncation. Verify interpolated sample metrics against the analytic fit and refine as needed. A short coarse array is suitable only as a fixture, not the production plot-resolution policy.

Each curve exposes `t30_s`, `t90_s`, `t_peak_s`, `virtual_origin_s` where applicable, and `t50_falling_s`, with voltage values at these landmarks. Include the target timing/crest metrics and each candidate's corrected metrics next to the plot data. Optional tolerance markers use the selected scalar test profile; a pointwise shaded voltage band must not be presented as a waveform-compliance or uncertainty envelope without a separately defined, validated basis.

Plot-bundle fields

Field	Contents
`bundle_id`, `run_id`, `context_id`	Stable identity and immutable scenario linkage
`time_s`, `units`, `time_origin`	Common time samples; voltage in V; declared origin
`target_curve`	Target metrics, `voltage_V` samples, `target_reference` origin, fit metadata and annotations
`candidate_curves`	One entry per returned candidate: candidate ID/rank, ML-corrected metrics, `voltage_V` samples, baseline/correction record IDs, model version, origin, fit/check status and landmarks
`fit`	Reconstruction version/family, alpha/beta rates, root diagnostics, desired/re-extracted metrics, residual fitting error
`sampling`	Grid strategy, point count, horizon, refinement/quality tolerances and any truncation
`plot_views`	Overlay curve IDs, target/candidate comparison pairs, full and front/peak zoom domains, axis/display-unit suggestions
`availability`	`complete`, `partial` or `failed`, returned/requested candidate counts, per-curve failure and shortfall reasons

Every voltage array must have the same length as `time_s`; all values are finite. Backend values remain SI; the client may display µs and kV using explicit scale metadata. Large arrays may use a stable downloadable JSON/CSV reference, but the values must remain retrievable rather than only an image. CSV contains time and one voltage column per curve; export metric/provenance metadata alongside it.

Standalone numeric fixture

These are **mock ML-corrected candidate metrics** for contract testing, not predictions for the physical configurations described elsewhere. The seven-point arrays are deliberately coarse examples; full production sampling must satisfy the policy above. Candidate IDs do not establish recommendation order without a corresponding objective evaluation.


```
{
  "fixture_kind": "mock_metrics_and_reconstructed_samples",
  "units": {
    "time": "s",
    "voltage": "V"
  },
  "time_origin": "impulse_start",
  "time_s": [
    0,
    5e-07,
    1.2e-06,
    2e-06,
    5e-06,
    5e-05,
    0.0001
  ],
  "target_curve": {
    "origin": "target_reference",
    "metrics": {
      "front_s": 1.2e-06,
      "tail_s": 5e-05,
      "crest_V": 1425000
    },
    "voltage_V": [
      0.0,
      1038248.615,
      1376317.208,
      1424798.376,
      1373531.588,
      710204.8,
      341269.924
    ]
  },
  "candidate_curves": [
    {
      "candidate_id": "mock-candidate-1",
      "origin": "ml_parameter_reconstruction",
      "metrics": {
        "front_s": 1.21e-06,
        "tail_s": 5.02e-05,
```

```
          "crest_V": 1420000
        },
        "voltage_V": [
          0.0,
          1030240.19,
          1369837.549,
          1419715.701,
          1369293.249,
          709772.604,
          342003.83
        ]
      },
      {
        "candidate_id": "mock-candidate-2",
        "origin": "ml_parameter_reconstruction",
        "metrics": {
          "front_s": 1.18e-06,
          "tail_s": 4.95e-05,
          "crest_V": 1430000
        },
        "voltage_V": [
          0.0,
          1050734.325,
          1384419.558,
          1429923.526,
          1377047.019,
          707457.844,
          337529.627
        ]
      },
      {
        "candidate_id": "mock-candidate-3",
        "origin": "ml_parameter_reconstruction",
        "metrics": {
          "front_s": 1.24e-06,
          "tail_s": 5.1e-05,
          "crest_V": 1415000
        },
        "voltage_V": [
          0.0,
          1013775.752,
          1359951.688,
          1414372.951,
          1366402.245,
          715404.219,
          348574.717
        ]
      }
    ]
  }
```


For the target fixture, the fitted rates are approximately `alpha=14657.593003682296 s^-1` and `beta=2474225.8641283913 s^-1`; the virtual origin is approximately `-2.2012703931501143e-7 s`. Re-extraction gives front `1.2e-6 s`, tail `5e-5 s` and crest `1425000 V`. Use the analytic crest rather than the maximum of this coarse fixture array.

Acceptance: exactly one target curve; one curve per returned candidate; correct candidate/metric linkage; numeric samples rather than image-only output; common array lengths and time origin; accurate fit/re-extraction for lightning and the selected switching convention; three candidate/target pairs plus one overlay; stable SI/display conversion; missing candidates and failed reconstructions exposed; no pointwise-ML or measured-trace claim. All tests can run with metric fixtures without physics/ML engines.

11. Public API contract

11.1 Endpoints

Endpoint	Request	Response
`GET /v1/profiles/equipment/{id}`	Profile ID	Every fixed constant, source, conflict and supported topology
`GET /v1/profiles/inventory/{id}`	Inventory ID	Saved example or registered front/tail catalogues, ratings, quantities, approvals

Endpoint	Request	Response
`POST /v1/inventories/validate`	Arbitrary inline inventory and quantity scopes	Normalised stock, errors, stage-dependent effective quantities/bounds and source manifest
`POST /v1/inventories`	Arbitrary inline inventory with source metadata	Registered immutable inventory ID/version/hash for subsequent run requests
`GET /v1/profiles/tests/{id}`	Test-profile ID	Targets, timing definition, tolerances and verification status
`GET /v1/models/{version}`	Model version	Three residual targets, baseline identity, extracted/selected feature manifests, fit report, measured/synthetic scope and support rules
`POST /v1/requests/validate`	`RunRequest`	Resolved context and full validation/constraint manifest
`POST /v1/physics/inverse`	Context plus stage count	Ideal proposal, charge derivation, flags
`POST /v1/assemblies/realise`	Context plus proposal	Exact actual wiring and DP/CP-SAT reports
`POST /v1/physics/evaluate`	Context plus realised candidate	Baseline, trace references, component stresses and constraints
`POST /v1/corrections/predict`	Context, realised candidate, matching baseline	Residuals, corrected values, support and explanation reference
`POST /v1/objectives/evaluate`	Context plus corrected candidate	All deviations, score, eligibility and ordering key
`POST /v1/runs`	`RunRequest`, idempotency key	`202` with run ID, resolved-context reference and status URL
`GET /v1/runs/{id}`	Run ID	Progress, counters, budgets and current top-three IDs
`GET /v1/runs/{id}/events`	Cursor	Ordered stage/proposal events; streaming may be added
`GET /v1/runs/{id}/candidates`	Pagination/filter	All eligible and rejected evaluated records
`GET /v1/runs/{id}/result`	Run ID	Complete ordered top-three, waveform plot-bundle reference, shortfall and stop report
`GET /v1/runs/{id}/waveforms`	Run ID and optional sampling/display policy	Target reference plus reconstructed curves for all returned candidates; common numeric grid and plot metadata
`POST /v1/waveforms/reconstruct`	Profile plus target/candidate metrics and sampling policy	Independently generated plot bundle with fit/extraction checks
`GET /v1/candidates/{id}`	Candidate ID	All nested input/output/constraint records
`GET /v1/candidates/{id}/generator-stages`	Candidate ID	Stage-indexed capacitor and resistor manifests
`GET /v1/candidates/{id}/explanations`	Candidate ID	Per-output attributions and physical heat evidence
`GET /v1/candidates/{id}/waveforms/{kind}`	`ml_parameter_reconstruction`, `physics` or `measured`	Numeric SI samples, curve origin, fit/extraction metadata and availability
`POST /v1/runs/{id}/cancel`	Run ID	Cancellation acknowledgement and preserved archive
`POST /v1/measurements`	Actual applied settings and measured trace	Measured shot record, quality checks and physical shot-rate verdict

No endpoint performs a physical shot. Measurement ingestion records acquisition time and distinguishes external observations from any future controlled laboratory acquisition. A separate acquisition workflow must enforce the 2/minute equipment limit; software candidate evaluation has separate budgets.

Synchronous module endpoints enable independent implementation, debugging and verification. They use the same validators as the orchestrated route. A module cannot accept a forged baseline/candidate combination: verify identity/version hashes and required derived fields. Large traces and complete histories are paginated or returned by references, never silently omitted.

11.2 Detailed stage and component response fields

The component contracts must use the following named groups; unavailable values are explicit `null` with statuses, not missing keys:

Object	Exposed fields
`GeneratorStage`	`generator_stage_index`, `active`, `charge_V`, `C_stage_F`, `stored_energy_J`, `front_assembly_id`, `tail_assembly_id`, expanded `component_ids`, `stage_voltage_utilisation`, `constraint_checks`, `equal_sharing_assumed`
`ComponentStress`	`component_id`, `generator_stage_index`, `role`, `type_id`, `nominal_ohm`, `resistance_interval_ohm`, `node_a`, `node_b`, `peak_voltage_V`, `peak_current_A`, `peak_power_W`, `pulse_energy_J`, `average_power_W`, `initial_temperature_K`, `estimated_peak_temperature_K`, `thermal_status`, `rating_limits`, `rating_margins`, `constraint_checks`, `trace_id`, `calculation_method`, `assumptions`
`ResidualOutput`	`output_name`, `unit`, `physics_value`, `residual_value`, `corrected_value`, `target_value`, `prediction_status`, `support_status`, `validated_interval` or `null`, `explanation_id`
`OutputExplanation`	`output_name`, `reference_residual`, `feature_contributions` with feature/value/unit/source/contribution, `aggregation_links`, `additivity_error`, `method_version`, `background_hash`, `unsupported_effects`, `physical_evidence_ids`, `explanation_status`
`SolverReport`	Solver/version, candidate-pool hash, search scope, statuses, objective/bound, time/part/state limits, used budget, scaling factors, selected option IDs, rejected-pair reasons and certificate scope
`PipelineStageRecord`	`pipeline_stage_name`, proposal/candidate/context IDs, input/output record IDs, start/end/duration, status, reason codes, constraints considered, checks skipped and cache indicator

A role-specific inventory type holds `pulse_voltage_limit_V`, `pulse_current_limit_A`, `pulse_energy_limit_J`, `average_power_limit_W`, `temperature_limit_K`, `R_theta_K_per_W`, `C_theta_J_per_K` and rating-evidence IDs. Unknown ratings remain `null`. Peak power alone is not a substitute for a duration-dependent pulse-energy rating. Profile-provided or measured resistance/thermal values must identify their applicable temperature/voltage/pulse conditions.

The stage endpoint expands all active stages even under uniform-stage assumptions. Identical computed values are labelled as modelled equal sharing, rather than separate stage measurements. Pipeline events and generator-stage responses use distinct field names to avoid ambiguity.

11.3 Request example

The following is a complete software diagnostic fixture, not a verified physical setup. Unknown auxiliary states/rating data remain visible after resolution.

```
{
  "schema_version": "1.0",
  "purpose": "diagnostic",
  "equipment_profile_id": "haefely_pdf_v1",
  "inventory": {"source": "registered", "profile_id": "project_front10_tail12_v1", "version": "1"},
  "impulse_profile_id": "lightning_project_v1",
  "system_max_voltage_V": 400000,
  "target": {"crest_V": 1425000},
  "load": {"model_id": "capacitive_v1", "C_F": 8.5e-10,
           "topology": "shunt_at_test_terminals", "terminal_connections": {"test": "hv", "return":
"ground"}},
  "setup": {"divider_C_F": 5e-10, "stray_C_F": 1.5e-10,
            "stray_L_H": 1.85e-5, "basic_capacitance_inclusion": "add_separately",
            "leads": {}, "return_path": {}, "geometry": {}},
  "environment": {"availability": "not_recorded", "values": {}},
  "expected_voltage_efficiency": 0.82,
  "charging_policy": {"kind": "fixed", "U_charge_V": 100000},
  "dp_tolerances": {"front_relative": 0.005, "tail_relative": 0.005},
  "search": {"max_proposals": 100, "max_forward_evaluations": 100,
             "max_ml_evaluations": 100, "dp_max_parts": 4,
             "dp_max_states": 100000, "dp_seconds": 10,
             "cp_sat_seconds": 2, "cp_pair_retries": 5,
             "repeat_window": 10, "archive_stagnation_window": 20,
             "wall_time_seconds": 300},
  "objective": {"policy": "worst_normalised_then_weighted",
                "weights": {"front": 1, "tail": 1, "crest": 1}},
  "model_version": "residual-tabpfn-v2-diagnostic-v1",
  "seed": 42
}
```

Budget and resistance-tolerance numbers above are explicit fixture policy choices, not equipment limits or tuned production defaults. Every implementation must permit configured budgets within service limits. This diagnostic request must not produce a falsely verified recommendation when its inventory/topology/model support is unresolved.

11.4 Result envelope

```
{
  "schema_version": "1.0",
  "run_id": "example-run",
  "status": "completed_with_shortfall",
  "context_id": "resolved-context-1",
  "stop_reason": "maximum_proposals",
  "requested_recommendations": 3,
  "returned_recommendations": 0,
  "shortfall_reason": "no_fully_eligible_candidate",
  "recommendations": [],
  "history_url": "/v1/runs/example-run/candidates",
  "waveform_plot_bundle_url": "/v1/runs/example-run/waveforms",
  "constraints_url": "/v1/requests/resolved-context-1/constraints",
  "counters": {"proposals": 100, "unique_realised": 12,
               "physics_evaluations": 12, "ml_evaluations": 0,
               "cache_hits": 88},
  "guarantee": "best distinct eligible setups among evaluated candidates only",
  "physical_recommendation_verified": false
}
```

A nonempty result uses full `CandidateEvaluation` records or lossless stable references as defined above, not only N/Rf/Rt summaries. Implement the context-constraint URL shown in the envelope as `GET /v1/requests/{context_id}/constraints`.

Errors return a stable code, stage, field path, offending value, applicable bound, source and retryability. Distinguish malformed input (`400`), semantic/profile conflict (`422`), incompatible run state (`409`) and server/component failure (`500`/`503`). A run finding no solution is a completed domain result, not a server exception.

11.5 API work package and acceptance

Implement route schemas and a mock job store without physics/ML dependencies. Contract-test arbitrary inline catalogue values/type counts, role-specific quantities, global stock and registered/inline source exclusivity. Publish OpenAPI and JSON schemas from the same types; do not maintain divergent field definitions. Contract-test all routes using fixture adapters. Test idempotent run creation, pagination, cancellation, version mismatch, rejected candidates and unknown constraints. Ensure full manifests remain retrievable after a run finishes. Plot-ready numeric data and metadata are in scope; actual frontend layout remains out of scope for this document.

12. Constraint responsibility matrix

Check	Context validation	DP/CP-SAT	Physics/stress	ML	Optimiser/archive	API
Protected source constants/profile conflicts	Resolve/report	Consume	Consume	Record identity	Freeze	Expose
Integer stage count / 2–12 bound	Validate range	Verify supplied N	Verify	Fixed input	Propose only supported integers	Expose N and check
200 kV/stage and 2,400 kV stack	Resolve	Candidate metadata where valid	Enforce actual charge	Cannot waive	Reject failed/unknown	Expose margins
2.5 kJ/stage and 30 kJ total	Resolve	Precomputed filters if valid	Enforce	Cannot waive	Reject failed/unknown	Expose each stage
Input inventories and independent stage caps	Validate arbitrary catalogues; derive stock availability	Enforce supplied quantities and per-role cap of 4	Check selected recipe	Assess realised feature support	Freeze input; derive bounds from actual stock	Expose submitted/effective counts, scopes and hash
Approved wiring/stage topology	Resolve	Select permitted	Check solver applicability	Baseline identity	Reject failed/unknown	Expose exact netlist

Check	Context validation	DP/CP-SAT	Physics/stress	ML	Optimiser/archive	API
		options				
Per-resistor ratings/thermal limits	Resolve evidence	Candidate-specific filters	Compute/check, unknown if insufficient	Feature evidence only	Reject failed/unknown	Expose each component
Auxiliary branch/basic capacitance	Allocate and resolve	Wiring metadata	Include active branch exactly once	Consistent features	Fixed scenario	Expose inclusion
Load/setup/environment	Validate model contract	Consume when ratings depend on it	Model supported topology	Feature/support checks	Fixed scenario	Expose all inputs
Waveform definitions/overshoot	Resolve profile	No direct nonlinear solve	Extract/check supported trace	Three metric corrections only	Corrected metric acceptance	Expose provenance
ML operating-domain support	Manifest	No inference of support	Baseline validity	Evaluate/flag	Exclude unsupported	Expose reasons
Resistance matching tolerance	Resolve policy	Enforce declared scope	Use actual values	Use actual values	Handle failure	Expose mismatch
Physical 2/minute shot rate	Resolve	Not software-search limit	Relevant acquisition workflow	Record measured conditions	Not ML iteration cap	Acquisition/measurement report
Search budgets/stagnation	Resolve	Enforce inner budgets	Count evaluations	Count predictions	Enforce outer limits	Expose counters/stop

A hard check must be enforced by its responsible deterministic component; including it in an ML feature vector is not enforcement. Unknown fixed inputs are propagated with their unresolved status, never replaced by guessed values.

13. Independent task completion and integration checks

An AI agent assigned one work package must:

1. Read the global contracts, its own work package and the relevant constraint rows.
2. Implement against the included fixtures and explicit interface protocols.
3. Provide typed/schema-valid outputs, documented failure codes and local tests.
4. Use adjacent-module mocks when necessary; mark every mock result accordingly.
5. List unresolved real-equipment/data requirements without pretending the entire pipeline is blocked from implementation.
6. Deliver a versioned adapter, reproducible local tests and contract evidence. Do not modify another module's contract silently.

Work package	Independent completion evidence
A: Context/profile	Resolved fixed constants, unit conversion, immutable context, source conflicts
B: Physics	Calculator benchmark, L=0 analytic reference, ODE convergence tests when available, extraction failures
C: Assembly	Exact example tree/netlist, bounded DP proof scope, independent quantities, CP-SAT status/rounding tests
D: Stress/correction	Per-leaf energy fixture, residual addition, baseline-version checks, unknown heat explanation, feature manifest
E: Optimisation	Mock SMAC/evaluator history, ordering, three distinct archive entries, deterministic budgets
F: Orchestrator	Mock end-to-end stage trace and all failure branches
G: Serialisation/API	Full candidate/constraint manifests, mock route contracts, cancellation/pagination
H: Plot data	Target plus three reconstructed curve fixtures, metric re-extraction, SI sample arrays, partial-result handling

Integration checks require consistent candidate IDs and immutable context across every stage; residual labels regenerated for the chosen baseline; actual rather than required resistances used in physics/ML; no combined inventory limit; all physical source limits enforced; honest support/thermal status; no duplicate top-three entries; no predicted labels used as measurements.

An implementation can pass software contracts while remaining unvalidated on hardware. Release reports must separate `contract_tests_passed`, `physics_reference_verified`, `inventory_hardware_verified`, `model_lab_validated`, `standard_extraction_verified` and `physical_recommendation_eligible`. Do not collapse them into one “validated” flag.

14. Provenance and remaining verification

Local source material:

- Working architecture: per-stage uniform configuration, independent stock, optimisation cycle and top-three decision.
- Physics formula book: equivalent circuit, inverse/forward formulas, normalised worst-error objective and workbook fixtures.
- Problem statement, especially pages 3–4: fixed generator data, variable setup and measured-data validation requirements.
- Reference workbook: complete calculator inputs, proposed catalogues and synthetic dataset.
- Existing nominal DP prototype, front catalogue, tail catalogue: reusable starting material, not a complete ratings-aware CP-SAT implementation.

The two supplied screenshots select SMAC3 and DP plus OR-Tools CP-SAT. Their claims about ratings apply only when ratings are represented as valid discrete constraints or precomputed verified options. Their shared-stock wording is superseded by the independent inventory requirement.

Outstanding equipment/data evidence: verified generator-side versus load-side tail connection; approved active stage arrangements; charging-resistor conflict; external branch presence/values; basic-capacitance location; actual supplied catalogue approvals and pulse ratings; capacitor transient/reverse limits; thermal parameters and initial temperature; measured environmental/load coverage; validated waveform extraction for the chosen standard. Record these as explicit profile/model evidence requirements.

This document creates implementation tasks and contracts. It does not claim that the specified engines, CP-SAT selection, residual training or API have already been implemented or validated.